阿里云 × CLOUD NATIVE COMPUTING FOUNDATION

云原生技术公开课

第 30 讲

理解 RuntimeClass 与使用多容器运行时

贾之光（甲卓） 阿里巴巴高级开发工程师

1

RuntimeClass 需求来源

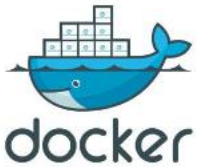
2

RuntimeClass 功能介绍

3

多容器运行时示例

Kubernetes 容器运行时演进过程



2014年6月
Kubernetes 开源

Docker 是 Kubernetes 唯一的容器运行时



Kubernetes v1.3

rkt 合入 Kubernetes，成为第二个容器运行时





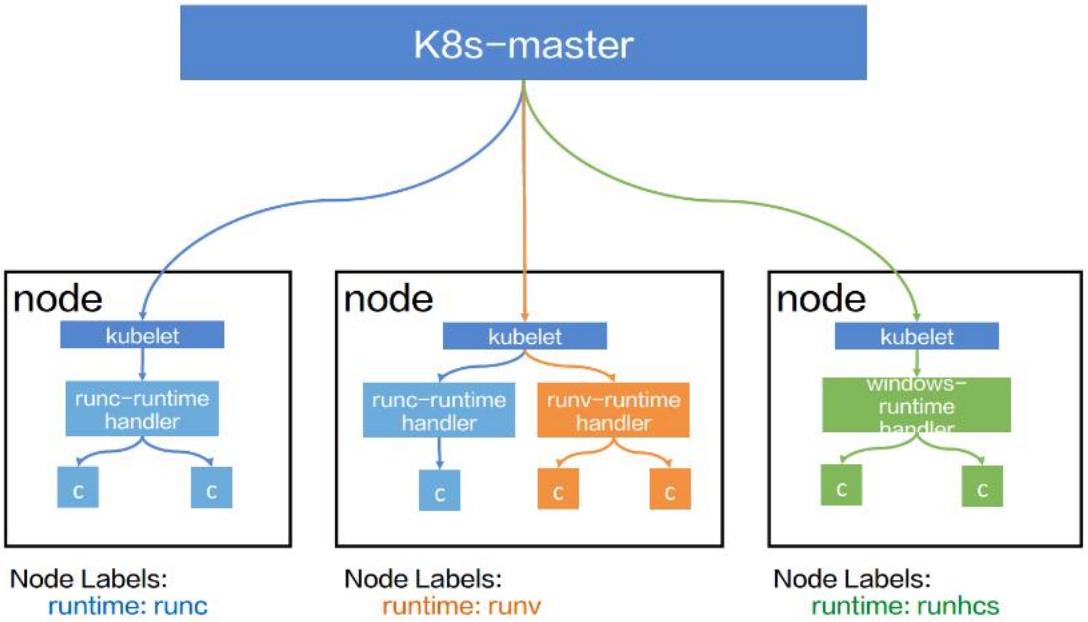
Kubernetes v1.5

Kubernetes 推出了 CRI，出现了 kata、gVisor 等实现了 CRI 的容器运行时

支持多个容器运行时需要解决的问题

- 有哪些容器运行时?
- 如何为 Pod 选择合适的容器运行时?
- 如何保证 Pod 调度到指定的容器运行时节点?
- 如何统计容器运行时下各种资源的额外开销?

RuntimeClass（kubernetes >= v1.16）




```
apiVersion: node.k8s.io/
v1beta1
handler: runv
kind: RuntimeClass
metadata:
name: runv
scheduling:
  nodeSelector:
    runtime: runv
---
apiVersion: v1
kind: Pod
metadata:
  name: my-runv-ypod
spec:
  runtimeClassName: runv
  …
```

1

RuntimeClass 需求来源

2

RuntimeClass 功能介绍

3

多容器运行时示例

RuntimeClass 结构体定义

```
type RuntimeClass struct {
 metav1.TypeMeta
 metav1.ObjectMeta

 Handler string
 Overhead *Overhead
 Scheduling *Scheduling
}
```

RuntimeClass：Kubernetes v1.14 引入该结构体，它是全局域资源，用来表示一个容器运行时

Handler：Kubernetes v1.14 该字段，对应 CRI 配置中的某个容器运行时的 handler

Overhead：Kubernetes v1.16 引入该字段，表示 Pod 的额外资源开销

Scheduling：Kubernetes v1.16 引入该字段，用于把 Pod 调度到支持该 RuntimeClass 的节点

RuntimeClass 资源定义例子

```yaml
apiVersion: node.k8s.io/v1beta1
kind: RuntimeClass
metadata:
  name: myclass
handler: myhandel
```

apiVersion：RuntimeClass 资源的 API 分组和版本

kind：RuntimeClass 资源类型

metadata.name：RuntimeClass 资源对象名称，Pod 可以引用该字段指定容器运行时

handler：表示 CRI 配置中的容器运行时的 handler 名称，最终由该 handler 来处理 Pod

Pod 引用 RuntimeClass 资源例子

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
 runtimeClassName: myclass
```

该 Pod 通过 spec.runtimeClassName 字段指定了 myclass 这个 RuntimeClass 资源。

最终 Pod 会被 myclass 对应的容器运行时处理

Scheduling 结构体定义

```
type Scheduling struct {
 NodeSelector map[string][string]
 Tolerations *[]core.Toleration
}
```

NodeSelector ：支持该 RuntimeClass 的节点上应该有的 label 列表。Pod 引用该RuntimeClass 后，RuntimeClass admission 会把该 label 列表与 Pod 中的 label 列表做合并。

注意：RuntimeClass 并不会自动为 Node 设置 label，需要用户在使用前提前设置好

Tolerations ：该 RuntimeClass 的容忍列表。 Pod 引用该 RuntimeClass 后，RuntimeClass admission 会把该 toleration 列表与 Pod 中的 toleration 列表做合并

为什么引入 Pod Overhead?



Pod Overhead 结构体定义

```
type Overhead struct {
 PodFixed core.ResourceList
}

type ResourceList map[ResourceName]resource.Quantity
```

PodFixed：各种资源的占用量，比如 CPU、内存的占用量

Pod Overhead 使用场景和使用限制

- **Pod 调度**：Overhead 与 容器的 requests 的累加值会作为 Pod 调度最小资源值
- **ResourceQuota**：Overhead 会被统计到资源限额中
- **Kubelet Pod 驱逐**：Overhead 会被统计到节点已使用资源中

Pod Overhead 还有一些使用限制和注意事项：

- Pod Overhead 最终会永久注入到 Pod 内并且不可更改。即便是 RuntimeClass 被删除或更新也是如此
- Pod 的 PodSpec.Overhead 只能由 RuntimeClass admission 自动注入，不可手动添加或更改，否则会被拒绝
- HPA 和 VPA 基于容器级别指标数据做聚合，Pod Overhead 不会对他们有影响

1

RuntimeClass 需求来源

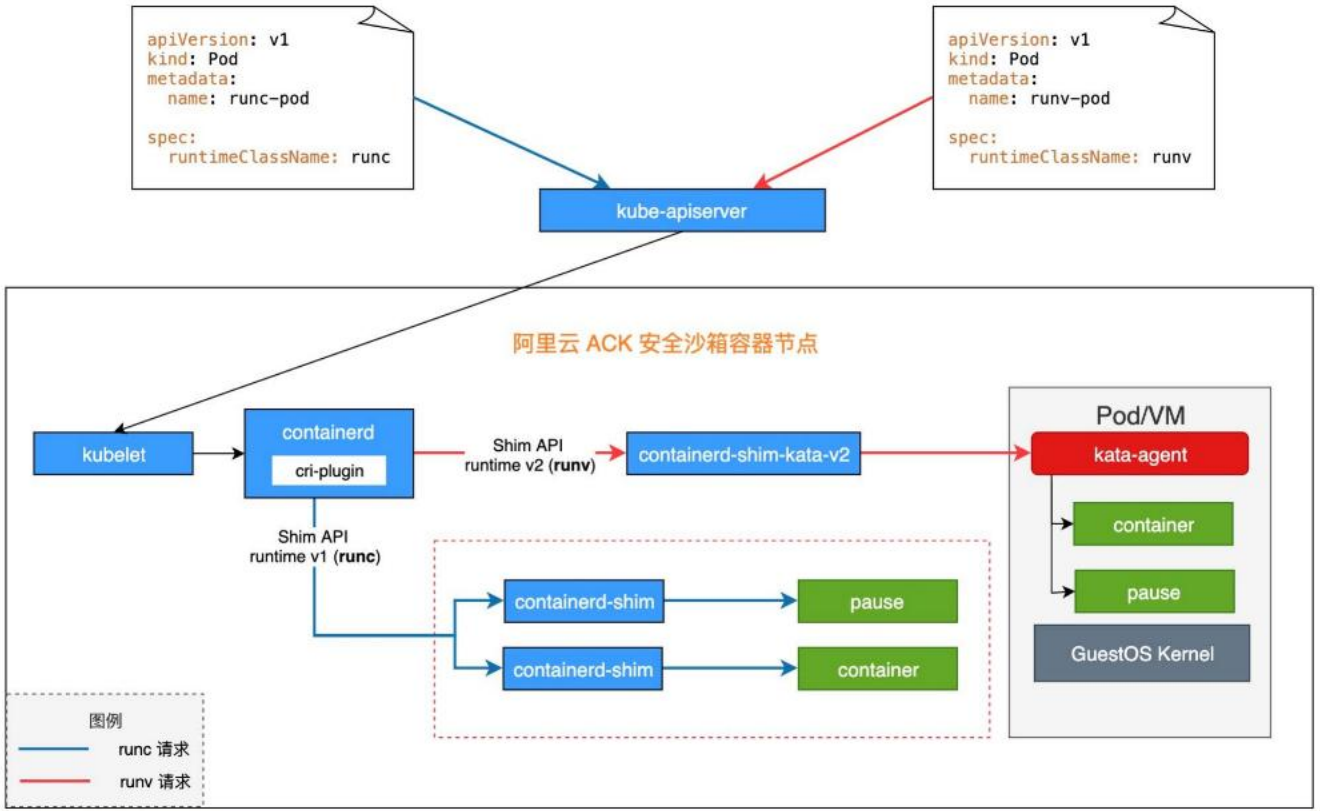
2

RuntimeClass功能介绍

3

多容器运行时示例

多容器运行时示例


apiVersion: v1
kind: Pod
metadata:
name: runc-pod
spec:
runtimeClassName: runc
apiVersion: v1
kind: Pod
metadata:
name: runv-pod
spec:
runtimeClassName: runv
kube-apiserver
阿里云 ACK 安全沙箱容器节点
kubelet
containerd
cri-plugin
Shim API
runtime v2 (runv)
containerd-shim-kata-v2
Pod/VM
kata-agent
container
pause
GuestOS Kernel
Shim API
runtime v1 (runc)
containerd-shim
containerd-shim
pause
container
图例
runc 请求
runv 请求

配置 containerd

```
# file:///etc/containerd/config.toml
[plugins.cri.containerd]
  [plugins.cri.containerd.default_runtime]
    runtime_type = "io.containerd.runtime.v1.linux"
  [plugins.cri.containerd.runtimes.runc]
    runtime_type = "io.containerd.runtime.v1.linux"
    ...
  [plugins.cri.containerd.runtimes.runv]
    runtime_type = "io.containerd.kata.v2"
    ...
```

- [plugins.cri.containerd.runtimes.runc]：runc 容器运行时配置
- [plugins.cri.containerd.runtimes.runv]：runv 容器运行时配置
- [plugins.cri.containerd.default_runtime]：如果 Pod 没有指定 RuntimeClass，默认使用 runc 容器运行时

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

创建 runc 和 runv RuntimeClass 资源

```yaml
apiVersion: node.k8s.io/v1beta1
kind: RuntimeClass
metadata:
  name: runc
handler: runc
---
apiVersion: node.k8s.io/v1beta1
kind: RuntimeClass
metadata:
  name: runv
handler: runv
```

```
[root@                    ~]# kubectl get runtimeclass
NAME   CREATED AT
runc   2019-11-18T03:24:24Z
runv   2019-11-18T03:24:24Z
[root@iZbp110r7rr727xhgqtt42Z ~]#
```


阿里云 CLOUD NATIVE COMPUTING FOUNDATION

创建 runc Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    name: runc-pod
  name: runc-pod
  namespace: demo
spec:
  runtimeClassName: runc
  containers:
    - image: nginx
      imagePullPolicy: IfNotPresent
      name: runc-pod
      ports:
        - containerPort: 8080
          protocol: TCP
  dnsPolicy: ClusterFirst
```


阿里云 CLOUD NATIVE COMPUTING FOUNDATION

创建 runv Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    name: runv-pod
  name: runv-pod
  namespace: demo
spec:
  runtimeClassName: runv
  containers:
    - image: nginx
      imagePullPolicy: IfNotPresent
      name: runv-pod
      ports:
        - containerPort: 8080
          protocol: TCP
  dnsPolicy: ClusterFirst
```


阿里云 CLOUD NATIVE COMPUTING FOUNDATION

课程总结

- RuntimeClass 是 Kubernetes 一种内置的全局域资源，主要用来解决多个容器运行时混用的问题
- RuntimeClass 中配置 Scheduling，可以让 Pod 自动调度到运行了指定容器运行时的节点上。但前提是节点上的 label 列表需要用户提前手动配置好
- RuntimeClass 中配置 Overhead，可以把 Pod 中业务运行所需以外的开销统计进来，让调度、ResourceQuota、Kubelet Pod 驱逐等更行为更准确

谢谢观看

THANK YOU